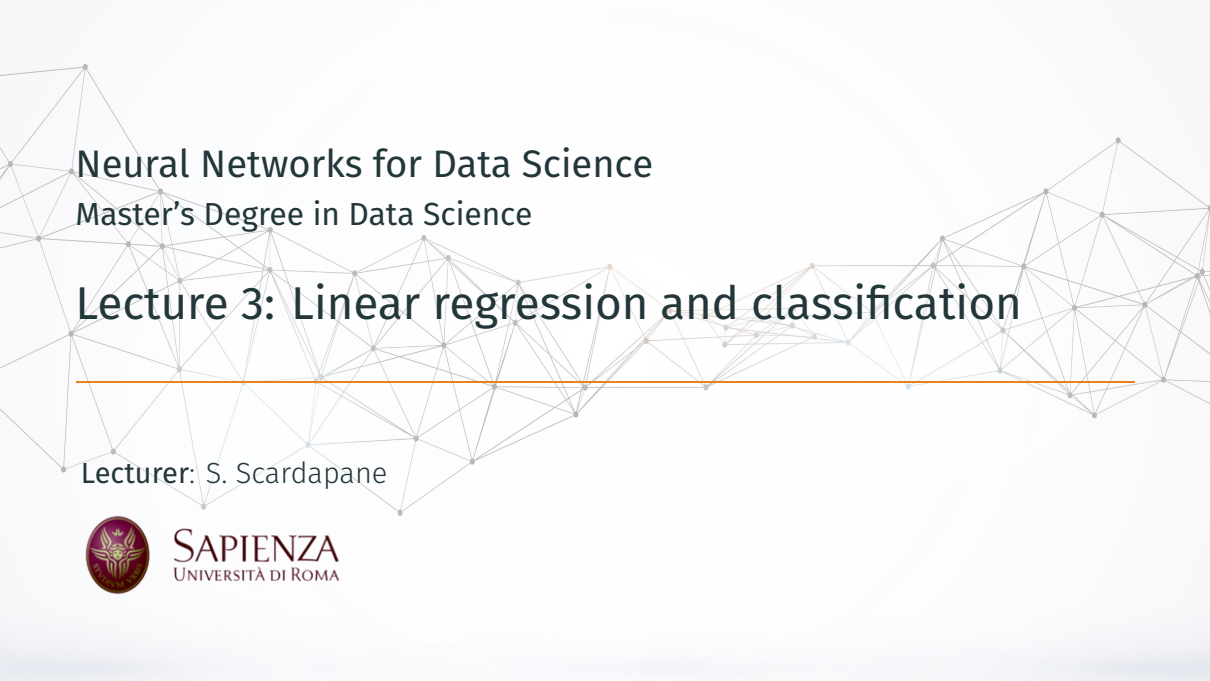




Neural Networks for Data Science

Master's Degree in Data Science

Lecture 3: Linear regression and classification

Lecturer: S. Scardapane



SAPIENZA
UNIVERSITÀ DI ROMA

Supervised learning

Setup and data splits

A supervised dataset is a set of N input-target pairs:

$$\mathcal{S} = \{(x_i, y_i)\}_{i=1}^N . \quad (1)$$

A model f_{θ} maps an input x to a prediction $\hat{y}$:

$$\hat{y} = f_{\theta}(x) . \quad (2)$$

We fit θ on $\mathcal{S}$ and evaluate the model on examples that were not used for training.

We split the available examples into three disjoint sets:

$$\mathcal{S} = \mathcal{S}_{\text{train}} \dot{\cup} \mathcal{S}_{\text{val}} \dot{\cup} \mathcal{S}_{\text{test}} . \quad (3)$$

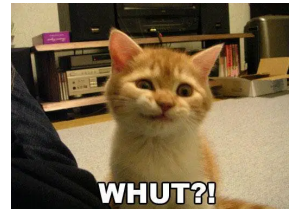
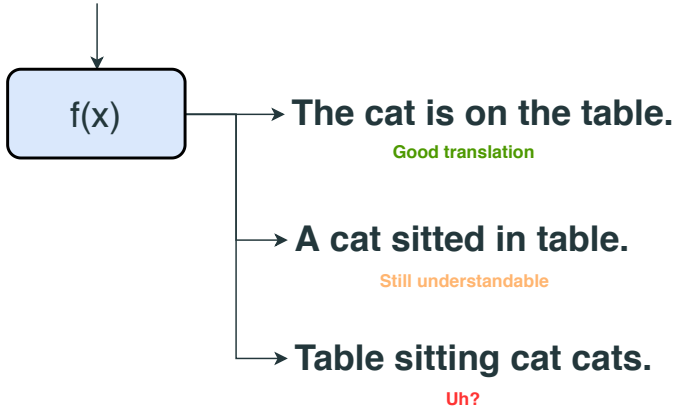
- ▶ $\mathcal{S}_{\text{train}}$ is used to fit the parameters θ .
- ▶ $\mathcal{S}_{\text{val}}$ is used to select the model and its hyper-parameters.
- ▶ $\mathcal{S}_{\text{test}}$ is used for the final evaluation.

We usually assume independent samples from the same unknown distribution $p(x, y)$ (*identically and independently* distributed, i.i.d.). If the deployment distribution differs, we have a **distribution shift**.

Supervised learning

Loss functions

"Il gatto è sul tavolo."



Given a point x , a desired value y , and a prediction $\hat{y} = f(x)$, we formalize its quality with a **loss function** $l(y, \hat{y})$, such that:

1. Low value of loss: good approximation;
2. High value of loss: poor approximation.

In this way, learning becomes a problem of minimizing a certain loss quantity that we designed. Importantly, the loss function should be a scalar, vary gradually, and (as we will see) be **differentiable**.

The **expected loss** (risk) of a function f is:

$$f^*(x) = \arg \min \left\{ \mathbb{E}_{p(x,y)} [l(y, f(x))] \right\} . \quad (4)$$

The expected risk is uncomputable, but can be approximated via **empirical risk minimization**:

$$f^*(x) = \arg \min_f \left\{ \frac{1}{n} \sum_{i=1}^n l(y_i, f(x_i)) \right\} . \quad (5)$$

The gap between the two approaches is called **generalization gap**.

Consider the following algorithm (a variation of 1-NN):

$$f(x) = \begin{cases} y & \text{if } (x, y) \in \mathcal{S}_n \\ \bar{y} & \text{otherwise} \end{cases} . \quad (6)$$

This has 0 training error by construction, but a very large test error: this is an example of **overfitting**. We will see that (large) neural networks have many counter-intuitive properties when it comes to generalization.

To begin our exploration of supervised learning, we will make a few simplifying assumptions:

- ▶ The input $\mathbf{x}$ is a vector of shape d .
- ▶ The output y is a single real number.

There are three interesting cases: when $y \in \mathbb{R}$, it is a **regression** task; when $y \in \{0, \dots, c - 1\}$, it is a **classification** task; when $c = 2$, it is a **binary classification** task.

Supervised learning

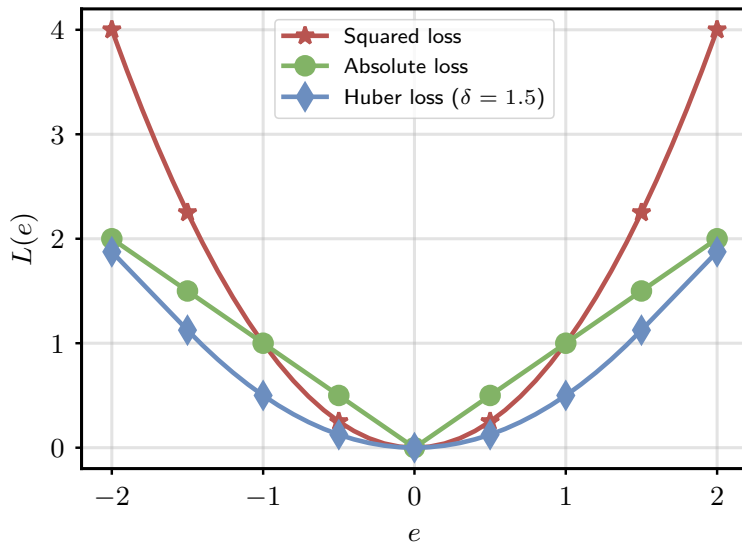
Building a loss function

Building a loss function can be done empirically: considering regression, for example, the prediction error $(y - \hat{y})$ is a reasonable quantity to penalize.

Since most times we do not care about the sign of the error, these are all valid choices:

$$\underbrace{(y - \hat{y})^2}_{\text{squared loss}}, \quad \underbrace{|y - \hat{y}|}_{\text{absolute loss}}, \quad \underbrace{(\log(1 + y) - \log(1 + \hat{y}))^2}_{\text{squared log loss}} \quad (7)$$

Is there a principled way to make the choice?



Let us assume that $f(x)$ parameterizes a *probability distribution* $\hat{p}(y | f(x))$ over the possible outputs y . We are now choosing $\hat{p}(\cdot | \cdot)$ instead of $l(\cdot, \cdot)$, which can feel more natural.

For example, for regression we can assume a Gaussian shape:

$$\hat{p}(y | f(x)) = \mathcal{N}(y | f(x), \sigma^2), \quad (8)$$

where the model predicts the center of a Gaussian distribution with fixed variance (hyper-parameter).

The probabilistic formulation also provides a principled way to interpret training by maximizing the **likelihood** (or log-likelihood) of the model (assuming the elements of the dataset are i.i.d.):

$$f^*(x) = \arg \max \prod_{(x_i, y_i)} \hat{p}(y_i | f(x_i)) = \arg \min \sum_{(x_i, y_i)} -\log \hat{p}(y_i | f(x_i)). \quad (9)$$

Under the Gaussian assumption in (8), we obtain $-\log \hat{p}(y_i | f(x_i)) \propto (y_i - f(x_i))^2$, i.e., we should optimize the squared loss!

Linear models

Linear models for regression

A linear model f is defined as:

$$f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle = \mathbf{w}^T \mathbf{x} = \sum_j w_j x_j, \quad (10)$$

where $\mathbf{w}$ is a vector of adaptable parameters.

This model is fundamental in many disciplines, ranging from econometrics to statistics.

A more general formulation considers the inclusion of an **offset** (bias) $b \in \mathbb{R}$:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b. \quad (11)$$

Because we can always rewrite this as $\mathbf{w}^\top \bar{\mathbf{x}}$, with $\bar{\mathbf{x}} = [\mathbf{x}; 1]$, we can avoid writing the bias explicitly to simplify the notation.

Hint: Everytime we write a linear model, mentally add an offset term whenever needed.

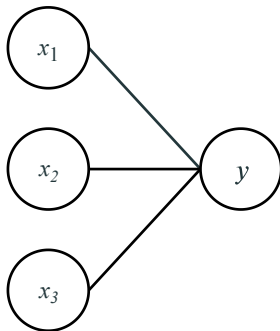


Figure 1: Each arrow represents a *linear* influence on the destination, which sums the results.

Combining the squared loss with a linear model results in the **least-squares** optimization problem:

$$\text{LS}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2. \quad (12)$$

We can vectorize LS as:

$$\text{LS}(\mathbf{w}) = \frac{1}{n} \left\| \underset{(n)}{\mathbf{y}} - \underset{(n,d)}{\mathbf{X}} \underset{(d)}{\mathbf{w}} \right\|^2, \quad (13)$$

where $[\mathbf{X}]_i = \mathbf{x}_i$ and $[\mathbf{y}]_i = y_i$.

Linear models

Solving the LS problem

LS is a convex problem, with a simple gradient (**normal equations**):

$$\nabla \text{LS}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}). \quad (14)$$

LS is special in the sense that $\nabla \text{LS}(\mathbf{w}) = 0$ is a linear equation that can be solved explicitly:

$$\mathbf{w}^* = \underset{(d,d)}{(\mathbf{X}^\top \mathbf{X})^{-1}} \underset{(d,n)(n)}{\mathbf{X}^\top} \underset{(n)}{\mathbf{y}} = \underset{(d,n)}{\mathbf{X}^\top} \underset{(n,n)}{(\mathbf{X}\mathbf{X}^\top)^{-1}} \underset{(n)}{\mathbf{y}}. \quad (15)$$

The terms $\underset{(d,d)}{(\mathbf{X}^\top \mathbf{X})^{-1}} \underset{(d,n)(n)}{\mathbf{X}^\top}$ and $\underset{(d,n)}{\mathbf{X}^\top} \underset{(n,n)}{(\mathbf{X}\mathbf{X}^\top)^{-1}}$ are called the **pseudoinverses** of $\mathbf{X}$, and they require the corresponding matrices to be invertible (full rank).

Numerical problems in the inversion of $(\mathbf{X}^\top \mathbf{X})$ can be solved by adding a small amount of ℓ_2 **regularization** (**ridge regression**):

$$\text{LS-REG}(\mathbf{w}) = \text{LS}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2, \quad (16)$$

for some $\lambda > 0$. This makes the problem *strictly* convex and forces the solution to be contained in a ball of given radius, modifying the gradient and the explicit solution as:

$$\nabla \text{LS-REG}(\mathbf{w}) = \nabla \text{LS}(\mathbf{w}) + \lambda \mathbf{w}. \quad (17)$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (18)$$

where $\mathbf{I}$ is the identity matrix of appropriate shape.

Generating some data:

```
1 # Linear model with unknown coefficients
2 X = tf.random.normal((10, 5))
3 y = X @ tf.random.normal((5, 1))
```

Computing a linear model:

```
1 w = tf.random.normal((5, 1))
2 yhat = X @ w # (10, 1)
```

Computing the objective function:

```
1 mse = tf.reduce_mean((y - yhat)**2)
```

Explicit solution (numerically unstable):

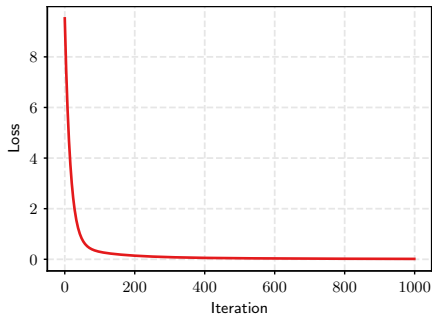
```
1 wopt = tf.linalg.inv(tf.transpose(X) @ X) @ tf.transpose(X) @ y
```

Explicit solution (better numerical conditioning):

```
1 wopt = tf.linalg.solve(tf.transpose(X) @ X, tf.transpose(X) @ y)
```

Simple implementation of gradient descent:

```
1 for i in range(15000):  
2     # Note the sign: the derivative has a minus!  
3     w = w + 0.001 * tf.transpose(X) @ (y - X @ w)
```



Matrix multiplication $\mathbf{A} \mathbf{B}$ has complexity $\mathcal{O}(abc)$, while matrix inversion has complexity (roughly) cubic.

For large models and datasets, we want algorithms that can scale linearly in both n and d : we will see that gradient descent can satisfy this property.

Linear models

Linear models for classification

In **classification**, y is an integer $\{0, \dots, c - 1\}$, such that $y_i = j$ means that $\mathbf{x}_i$ is of class j .

For example, with $c = 3$ we might have:

- ▶ $y = 0$: the email is spam;
- ▶ $y = 1$: the email is legit;
- ▶ $y = 2$: the email is dubious.

Solving these as regression tasks is generally not an optimal choice: among other things, it is not guaranteed that classes have a definite ordering.

A common solution is to predict a **probability distribution** over the classes.

A vector **a** belongs to the **probability simplex** Δ_c if:
(c)

$$\sum_i [a]_i = 1, \quad [a]_i \geq 0. \quad (19)$$

If $f(\mathbf{x}) = \hat{\mathbf{y}} \in \Delta_c$, we can interpret it as a **categorical probability distribution**, e.g., we can select the class with highest probability as:

$$\text{class} = \arg \max_i [\hat{\mathbf{y}}]_i. \quad (20)$$

Note that we *cannot* easily predict an integer with our models, because it would require some form of threshold operation which is not compatible with gradient descent (gradient zero almost everywhere).

Predicting a probability distribution can be seen as a *soft* approximation to this problem.

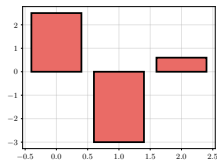
The **softmax** function maps any vector to the probability simplex:

$$[\text{softmax}(\mathbf{a})]_i = \frac{\exp(a_i)}{\sum_j \exp(a_j)} \quad (21)$$

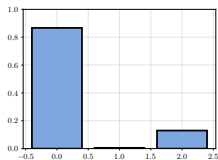
The numerator ensures that all outputs are positive, while the denominator ensures that the final vector sums to 1. It can be seen as a soft approximation to the argmax (a better name is in fact *softargmax*).

If desired, we can control the approximation with an additional hyper-parameter τ called **temperature**:

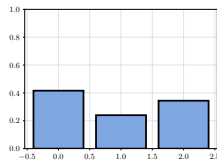
$$[\text{softmax}(\mathbf{a})]_i = \frac{\exp(a_i/\tau)}{\sum_j \exp(a_j/\tau)} \quad (22)$$



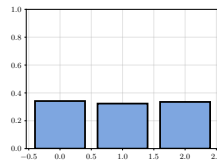
(a) Logits



(b) $\tau = 1$



(c) $\tau = 10$



(d) $\tau = 100$

Figure 2: Input logits (red) and softmax outputs with three different temperature values.

Our linear model for classification becomes:

$$f(\mathbf{x}) = \underset{(c)}{\text{softmax}}\left(\underset{(c,d)}{\mathbf{W}} \underset{(d)}{\mathbf{x}}\right) \quad (23)$$

The pre-softmax values $\mathbf{W}\mathbf{x}$ are called the **logits** of the model. By explicitly writing the biases $\mathbf{b}$:

$$f(\mathbf{x}) = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b}). \quad (24)$$

In order to compare the predictions with the ground truth, we encode our targets using a **one-hot encoding**. Given a pair $(\mathbf{x}, \mathbf{y})$:

$$y_i = \begin{cases} 1 & \text{if } \mathbf{x} \text{ is of class } i, \\ 0 & \text{otherwise.} \end{cases} \quad (25)$$

For example, with 3 classes $\{\text{cat}, \text{dog}, \text{other}\}$:

$$\text{cat} = [1, 0, 0] \quad \text{dog} = [0, 1, 0] \quad \text{other} = [0, 0, 1]. \quad (26)$$

This is a probability distribution putting all the **mass** on a single class.

Finally, we need a loss function l to compare two probability distributions.

The **cross-entropy** loss is defined for two vectors $\mathbf{y}, \hat{\mathbf{y}} \in \Delta_c$ as:

$$\text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \log(\hat{y}_i) . \quad (27)$$

The CE loss can be derived from the maximum likelihood principle under the assumption that $f(\mathbf{x})$ encodes a **categorical** distribution (try it!).¹

¹The equation for the categorical distribution in this case would be $p(\mathbf{y}|f(\mathbf{x})) = \prod_i [f(\mathbf{x})]_i^{y_i}$.

The CE loss is generic for *any* pair of vectors belonging to the probability simplex. However, in our case $\mathbf{y}$ is always a one-hot encoded vector.

Denote by t the index of the class, $t = \arg \max \mathbf{y}$, the CE loss simplifies since only one term in the summation will be different from 0:

$$\text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = -\log(\hat{y}_t). \quad (28)$$

Hence, CE can be interpreted as maximizing the probability of the true class at the expense of all other outputs.

A **logistic regression** is a linear model $f(\mathbf{x}) = \text{softmax}(\mathbf{W}\mathbf{x})$ trained by optimizing the cross-entropy:

$$\text{LR}(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \text{CE}(\mathbf{y}_i, f(\mathbf{x}_i)) . \quad (29)$$

It is not possible to solve the logistic regression problem explicitly. A linear model for classification has dc parameters (or $dc + c$ with biases).

A special case is **binary classification**, where $c = 2$. In this case, we can predict a single scalar value $f(\mathbf{x}) \in [0, 1]$ since:

$$f(\mathbf{x}) \quad \text{probability of class 1 ,} \quad (30)$$

$$1 - f(\mathbf{x}) \quad \text{probability of class 2 .} \quad (31)$$

The softmax function simplifies to the **sigmoid** function:

The sigmoid $\sigma(s) \in [0, 1]$ is defined as:

$$\sigma(s) = \frac{1}{1 + \exp(-s)} . \quad (32)$$

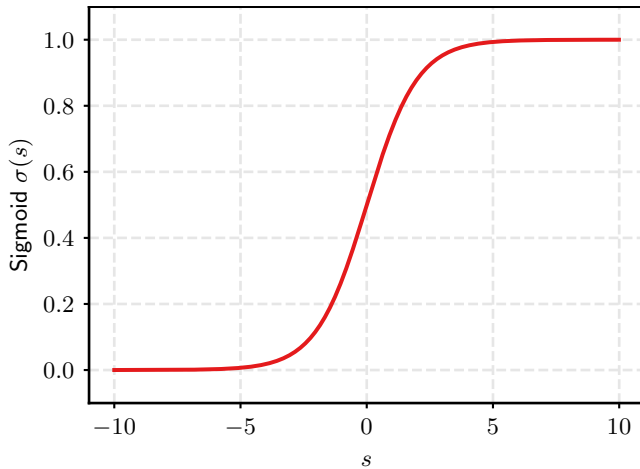


Figure 3: A visualization of the sigmoid function. Note that 0 and 1 are only approached asymptotically.

Combining everything, we obtain a binary version of the logistic regression algorithm:

$$\text{BIN-LR}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \left[\underbrace{-y_i \log(\sigma(\mathbf{w}^\top \mathbf{x}))}_{\text{Class 1}} - \underbrace{(1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}))}_{\text{Class 2}} \right] \quad (33)$$

In this case, we can obtain the most probable class from the model as:

$$\text{class} = \begin{cases} 1 & \text{if } \sigma(\mathbf{w}^\top \mathbf{x}) > 0.5, \\ 0 & \text{otherwise .} \end{cases} \quad (34)$$

By manually differentiating we obtain:

$$\sigma'(s) = \sigma(s)(1 - \sigma(s)). \quad (35)$$

Plugging this into the gradient computation we obtain:

$$\nabla \text{BIN-LR}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i) \mathbf{x}_i, \quad (36)$$

showing its similarity to the regression case.

```
1 from tensorflow.keras.metrics import *  
2  
3 # The one we have described up to now.  
4 categorical_crossentropy(ytrue, yhat)  
5  
6 # ytrue should contain the indexes of the classes instead of the  
7 # one-hot encodings.  
8 sparse_categorical_crossentropy(ytrue, yhat)  
9  
10 # Numerically-stable versions requiring the logits as inputs  
11 categorical_crossentropy(ytrue, yhat, from_logits=True)  
12 sparse_categorical_crossentropy(ytrue, yhat, from_logits=True)
```

Why a variant with logits in input? Note that the i th term of the cross-entropy wrt the logits $\mathbf{p}$ is given by:

$$-\log \left(\frac{\exp p_i}{\sum_j \exp p_j} \right) . \quad (37)$$

In practice, this can be *highly* unstable. However, it can be rewritten as:

$$-p_i + \text{logsumexp}(\mathbf{p}) , \quad (38)$$

where logsumexp is defined as $\text{logsumexp}(\mathbf{p}) = \log(\sum_i \exp p_i)$.

The reason this is important is that the logsumexp function is invariant in the following sense:

$$\text{logsumexp}(\mathbf{p}) = \log \left(\sum_i \exp(p_i - c) \right) + c, \quad (39)$$

where c is an arbitrary constant. By setting $c = \max(\mathbf{p})$, we can ensure numerical instabilities never occur.

In this sense, softmax can be interpreted as part of the model *or* as part of the loss; this is not an issue, since $\arg \max$ only cares about the relative ranking of the values, which is not changed by the softmax.

Linear models

Calibration and a probabilistic
formulation

A common misconception when doing classification is that $[f(x)]_i$ can be immediately interpreted as *the probability of pattern x being of class i* .

However, this is only true whenever the trained model satisfies:

$$p(y = i | x) = [f(x)]_i. \quad (40)$$

We say the model is well **calibrated**, but this must be checked manually.

Guo, C., et al.. On calibration of modern neural networks. ICML 2017.

To measure calibration, we keep a separate validation set, and we split the interval $[0, 1]$ into m equispaced bins (each of size $1/m$). Define:

- ▶ B_m the number of samples from the validation set, whose predicted confidence falls in bin m .
- ▶ p_m the average confidence of the network for that bin.
- ▶ a_m the average accuracy of the network for these elements.

Then, the **expected calibration error** (ECE) is given by:

$$\text{ECE} = \sum_i \frac{|B_i|}{n} |a_i - p_i| \quad (41)$$

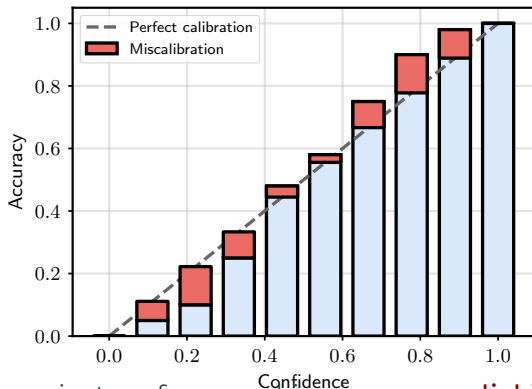


Figure 4: Plotting a_m against p_m for every bin gives us a **reliability plot** (from Guo et al., 2017).

This topic is important because more complex networks may be highly over (or under) confident, with many methods to improve it (temperature scaling, logit normalization, ...).

A simple (and popular) option is to decrease the weight given to ‘easy’ samples using a variant of cross-entropy call the **focal loss**:

$$\text{FL}_\alpha(\mathbf{y}, \hat{\mathbf{y}}) = -(1 - \hat{y}_c)^\alpha \log \hat{y}_c, \quad (42)$$

where $c = \arg \max \mathbf{y}$.

Mukhoti, J., et al., 2020. **Calibrating deep neural networks using focal loss**. *Advances in Neural Information Processing Systems*, 33, pp. 15288-15299.

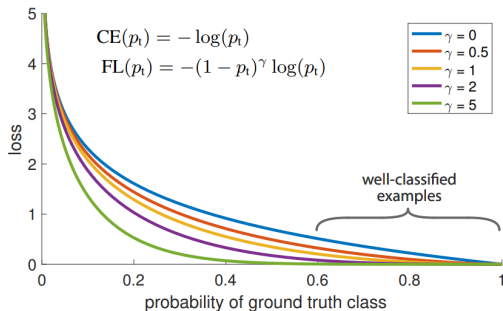


Figure 1. We propose a novel loss we term the *Focal Loss* that adds a factor $(1 - p_t)^\gamma$ to the standard cross entropy criterion. Setting $\gamma > 0$ reduces the relative loss for well-classified examples ($p_t > .5$), putting more focus on hard, misclassified examples. As

- ▶ **Chapters 3 and 4** of the book, along with the suggested exercises in Chapter 4.